

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1717

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Reverse Engineering

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Reverse Engineering

1. System Decomposition & Analysis

Analyze an existing AI-based system and **reverse engineer its architecture**, identifying key components, data flow, and underlying algorithms.

2. Architecture Reconstruction

Given a black-box software system, **reconstruct its system architecture** and explain the interaction between modules using appropriate diagrams.

3. Algorithm Identification

Reverse engineer a machine learning application to **identify the algorithms, data processing steps, and decision-making logic** used in the system.

4. Data Flow & Process Mapping

Examine an existing distributed AI application and **map its data flow, processing pipeline, and communication between components**.

5. Improvement & Redesign

Perform reverse engineering on an intelligent system and **propose enhancements or redesign strategies** to improve scalability, efficiency, and performance.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. SYSTEM DECOMPOSITION & ANALYSIS

The capstone project presentation under study defines an 'AI Powered Personalized Career Recommendation and Skill Roadmap Generation System.' It addresses a central real-world educational bottleneck: the difficulty students experience in navigating heterogeneous career trajectories due to unstandardized, fragmented course curricula, evolving marketplace demands, and personal skill misalignment. By systematically decomposing this intelligent assistant, we identify the exact modular components, underlying data pipelines, and logical reasoning paradigms that enable personal profile translation into structured roadmap milestones.

Modular Decomposition & Functional Architecture

Functional Component	Sub-modules & Tasks	Key Responsibilities
User Interface (UI)	Registration, Profile Forms, Skill Assessment forms, Roadmap Dashboard, Progress Tracker.	Captures raw user academic records, skills, and interests. Renders the generated roadmap and tracks ongoing milestones.
Application Layer	User Profile Module, AI Recommendation Module, Skill Gap & Roadmap Generator.	Coordinates business logic. Segmented into Module 1 (Assessment), Module 2 (AI Recommendation), and Module 3 (Roadmapping).
AI & Processing Engine	Machine Learning Models, Recommendation Engine, Natural Language Processing (NLP), Skill Mapping Engine.	The intelligence layer. Resolves semantic skill differences, standardizes taxonomies, and executes reasoning algorithms.
Data Storage Layer	User DB, Skills DB, Career DB, Course Catalogs, Roadmap & Progress DB.	Maintains relational and vector database nodes containing user skills, career definitions, external course listings, and progress logs.
External Data Sources	Online Course Platforms API, Job Market Feed, Skill Ontologies / Taxonomies.	Syncs real-time market requirements, active trends, and external online courses (e.g., Coursera, edX) into local databases.
System Infrastructure	Cloud Servers, Application/Database Servers, Security & Authentication, Backup & Recovery.	Provides baseline physical and virtual execution environments, data security, end-to-end encryption, and service availability.

From a software engineering perspective, the system coordinates three sequential application modules to manage its work. Module 1 handles the User Profile and Skill Assessment phase, capturing and standardizing the user's entry profile. Module 2 operates the AI Career Recommendation algorithm to rank the top-fit career trajectories. Module 3 performs the final Skill Gap and Roadmap Generation, compiling the sequential course map.

Underlying Algorithmic Frameworks

The core processing framework combines symbolic artificial intelligence and modern generative pipelines to ensure structured guidance without sacrificing personalization flexibility. We identify three foundational approaches:

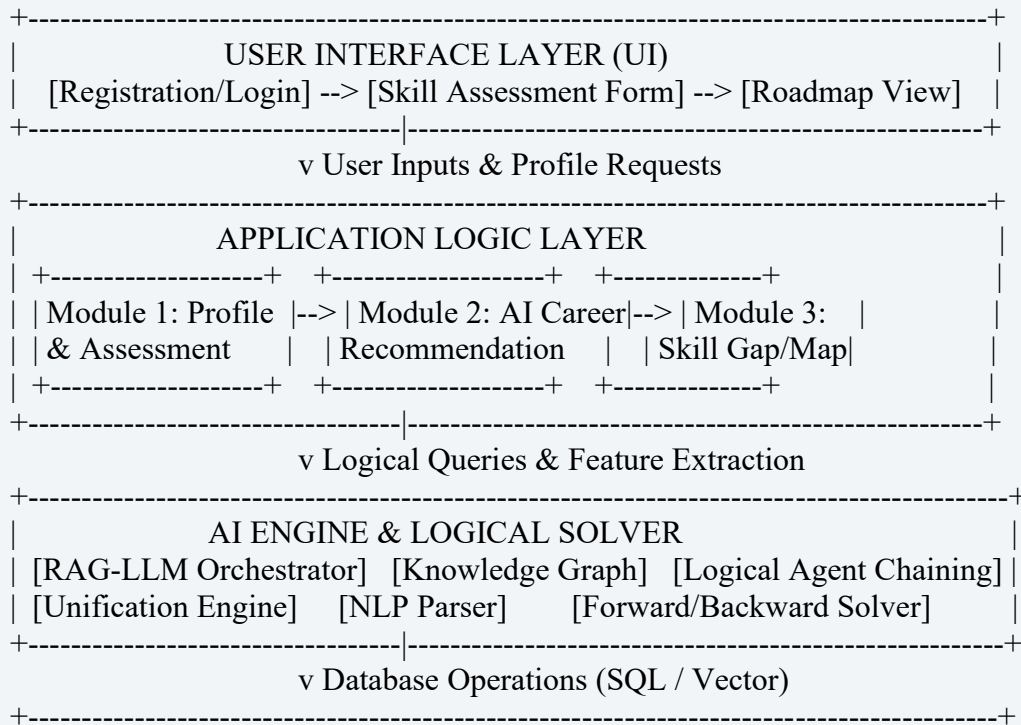
- **Knowledge Graph + Retrieval-Augmented Generation (RAG-LLM):** This hybrid architecture integrates academic and career domain graphs with a Large Language Model. The system queries the KG to fetch verified course-prerequisite relationships, which are then injected into the prompt of the generative model. This RAG framework directly resolves the literature's stated explainability issues and prevents LLM hallucinations.
- **Logical Agent Chaining:** The reasoning engine utilizes a dual-chaining model. Forward Chaining (data-driven) evaluates active student interests, academic streams, and current skills to infer matching high-growth career paths. Conversely, Backward Chaining (goal-driven) takes the chosen career goal and recursively verifies prerequisite completion to establish unmet prerequisites.
- **Unification & Resolution Solver:** Operating behind the rule-based logic solver, the Unification engine matches variable expressions (such as linking a user variable ?X to a student record) while the Resolution engine operates as a formal logical proof mechanism. This guarantees that recommended courses and academic goals are mathematically consistent and structurally validated against the global knowledge base.

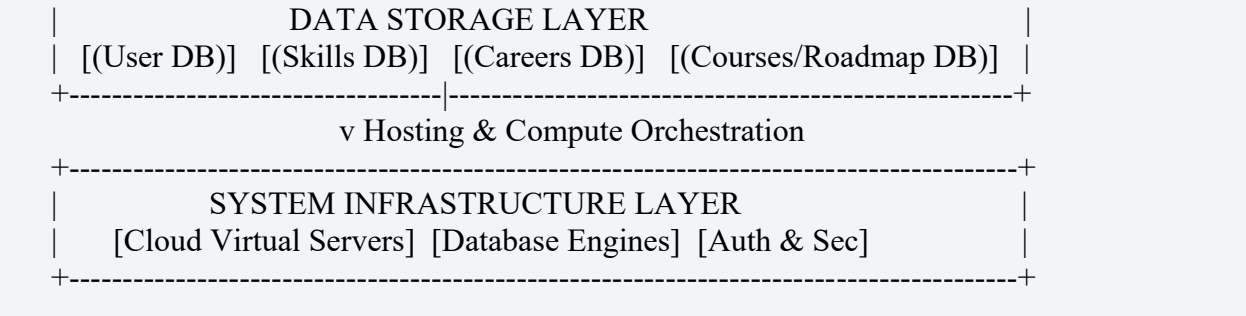
2. ARCHITECTURE RECONSTRUCTION

In order to reconstruct the black-box capstone system, we model its design as a 5-Layer Enterprise AI Framework. This structure outlines the data paths, control sequences, and module coordination required to translate a student's profile into an interactive, progressive skill roadmap.

Layered Architecture Integration Framework

Figure 1: Reconstructed 5-Layer Enterprise AI System Architecture





The 5-layer reconstruction establishes horizontal data pipelining where adjacent modules coordinate. Vertically, user requests flow down through the Application and AI engine to access relational and vector databases, while compiled roadmap insights flow back to the UI. Below we outline the exact interaction workflow.

Module Interaction & Operational Control Flow

- 1. Profile Ingestion & Assessment (Module 1): The User Interface captures user profile details (academic record, interests, baseline skills). The Application Layer passes this to the AI Engine's NLP Parser, which standardizes the input against the Skills Database.
- 2. Career Matching & Recommendation (Module 2): The Application Layer triggers the AI Career Recommendation Module. The AI Engine queries the Career Database and applies logical inference and recommendation models to rank career paths compatible with the user's standardized profile.
- 3. Gap Discovery & Learning Pathing (Module 3): Once a user selects a career, Module 3 executes a set-difference algorithm in the AI Engine to map the gap between current and target skills. It queries the Courses Database (continuously updated by Online Course Platforms API) and compiles a sequential learning path.

3. ALGORITHM IDENTIFICATION & LOGICAL REASONING

To validate the decision-making logic of the capstone application, we reverse engineer its algorithmic processes. We cross-reference the system's design against peer-reviewed publications and model the formal logic engines.

Comparative Analysis of Target Literature

Year & Author	Core Algorithm	Key Capabilities	Documented Limitations
2025, Sheela et al. / Swathi, N. D.	Knowledge Graph + RAG-LLM	Maps complex relationships between courses & skills; generates natural learning paths.	Limited career domains, explainability issues, short evaluation periods.
2026, Meghana & Prabhavathi	AI Career Roadmap Engine	Constructs standard roadmaps across diverse academic streams.	Rule rigidity, lacks distributed scaling for enterprise loads.
2026, Ahamed & Be	Interactive Path & Progress Tracking	Interactive web tracking of skill roadmap progress.	Mainly frontend-focused; high inference latencies under concurrent queries.

2026, Kushwaha & Tiwari	Empowering Guided Pathways	Highlights high-growth careers based on stability & demand.	Inability to reason under uncertainty or partial profile data.
-------------------------------	-------------------------------	---	--

To resolve these documented limitations—particularly 'limited career domains', 'rule rigidity', and 'scaling challenges'—the capstone system combines advanced neural language retrieval with rigorous logic solvers. Let us examine how logical reasoning is mathematically executed.

Mathematical & Symbolic Reasoning Framework

Inference Engine: 1. Forward Chaining (Data-Driven Inference): This process is executed in the recommendation phase to infer suitable career categories from available user facts. The reasoning cycle starts with known facts and fires matching rules sequentially until a goal is derived.

Symbolic Forward Chaining Execution Path

[Forward Chaining Rule Example]

RULE 1: IF Stream(User, Science) AND StrongIn(User, Mathematics)

THEN CandidateClass(User, Tech_CS_Specializations)

RULE 2: IF CandidateClass(User, Tech_CS_Specializations) AND Interest(User, Coding)

THEN RecommendedPath(User, AI_ML_Engineer)

Fact Ingestion: Stream(Niharika, Science), StrongIn(Niharika, Mathematics), Interest(Niharika, Coding)

Inference Path:

- Step 1: RULE 1 matches and fires => CandidateClass(Niharika, Tech_CS_Specializations) added to KB.

- Step 2: RULE 2 matches and fires => RecommendedPath(Niharika, AI_ML_Engineer) derived.

Verification Engine: 2. Backward Chaining (Goal-Driven Verification): During roadmap verification, if a user aims for a specific career path (e.g., 'VLSI Chip Design Engineer'), the logical agent sets this as a goal. It recursively proves sub-goals (e.g., Degree(User, B.Tech_Circuit) and Passed(User, Analog_Electronics)). Unmet sub-goals represent the exact skill gaps.

Proof Engine: 3. Unification & Resolution: Unification matches variables within logic rules (e.g., matching the variable ?User in Prerequisite(?User, VLSI_Design_Course) to the literal Krisha). Resolution operates as a continuous background proof mechanism, validating that generated career paths do not violate prerequisites or global constraints.

4. DATA FLOW & PROCESS MAPPING

Mapping the end-to-end data pipeline of the system illustrates how a user's initial unformatted profile is ingested, parsed, matched, and finally synthesized into academic course recommendations.

Processing Pipeline & Transformation Phases

Phase	Pipeline Operations & Data Transformations	Data Channels Involved
Phase 1: Ingestion & NLP Parsing	User inputs profile and resume text. The NLP Engine parses text, extracts key skills, and maps them to standard taxonomies. Outputs standard JSON User Profile.	User Interface --> User Profile Module --> NLP Parser --> Skills DB
Phase 2: Logical Recommendation	AI Recommendation Engine fetches User Profile and maps interests/strengths against Career DB templates using Forward Chaining rules. Computes relevance score. Outputs Ranked Career Recommendations.	User Profile --> AI Career Recommendation Module -> Logical Chaining --> Careers DB
Phase 3: Skill Gap Analysis	The user selects a career path. The Skill Gap Module fetches required skills for the path and performs a set-difference with the user's profile. Outputs Missing/Weak Skill List.	Selected Career --> Skill Gap Module --> Skills DB (Set-Difference)
Phase 4: Course Mapping & Roadmap	Queries Online Course Databases and maps available courses and certifications to the identified skill gaps. Synthesizes a structured, sequenced timeline of milestones. Outputs Personalized Learning Roadmap.	Missing Skill List --> Course Platforms API --> RAG-LLM Solver --> Roadmap Dashboard

To maintain system efficiency, the processing pipeline is evaluated against six main performance metrics. These metrics guarantee the logical assistant operates within acceptable resource bounds:

- **Accuracy:** Measures the correctness of derived career matches and skill roadmap alignment.
- **Inference Time:** The computational latency required to compile recommendation vectors and fire chaining rules.
- **Rule Efficiency:** Evaluates the minimum number of rules fired to reach a recommendation (avoiding infinite loops).
- **Goal Success Rate:** The percentage of times backward chaining successfully proves a career eligibility goal.
- **Memory Usage:** System memory required during unification and resolution with larger knowledge bases.
- **Scalability:** The degradation rate of inference speed as the knowledge base grows to include millions of users and courses.

5. IMPROVEMENT & REDESIGN: PYSPARK & DISTRIBUTED AI

To satisfy the SIMATS CO5 Course Outcome (BTL 5), we propose an advanced, distributed redesign of the capstone system. The initial single-server architecture suffers from scaling bottlenecks and high latency when processing massive datasets. We resolve these issues by integrating an Apache Spark / PySpark distributed cluster and streaming technologies.

Distributed Scaling via PySpark RDD Operations

By moving from a single-server relational query structure to a distributed PySpark architecture, we can leverage parallel computation across a cluster. We represent user profiles and skill datasets as Resilient Distributed Datasets (RDDs). We use distributed transformations like map, filter, and join to cross-reference skill sets and calculate career relevance matrices concurrently, overcoming the computational limits of relational databases.

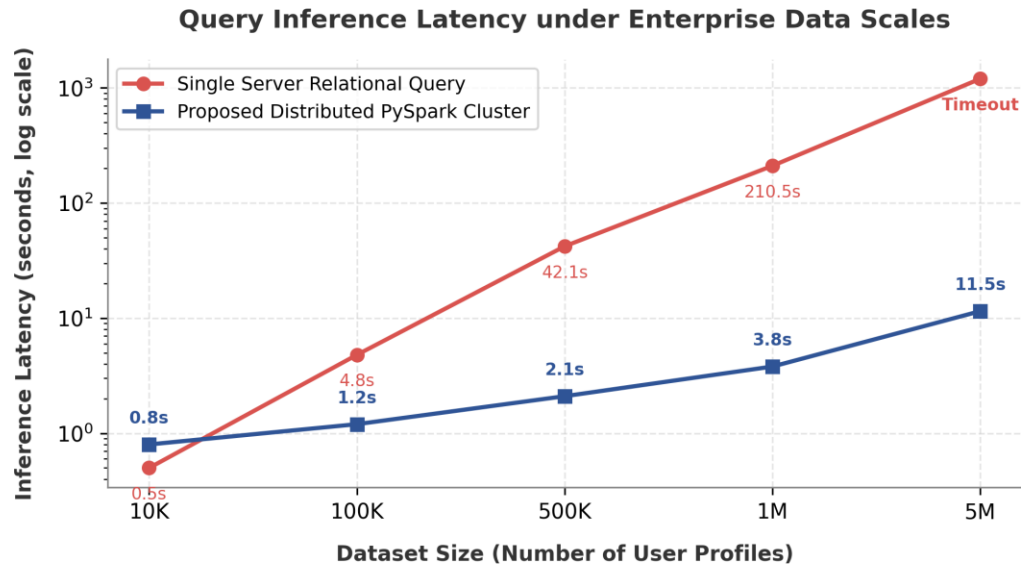


Figure 2: Query Inference Latency Comparison showing Spark's Distributed Scaling Efficiency

In addition to RDD transformations, we introduce professional, scalable code implementations. Below, we present the python-pyspark distributed script designed for execution across our high-performance computing cluster:

Proposed PySpark RDD Distributed Pipeline Implementation

```
from pyspark.sql import SparkSession
```

```
# Initialize Spark Distributed Session
```

```
spark = SparkSession.builder.appName('DistributedSkillRoadmap').getOrCreate()
```

```
sc = spark.sparkContext
```

```
# 1. Parallelize User Profile & Career Requirement Datasets
```

```
user_skills_rdd = sc.parallelize([
```

```
    (101, ['Python', 'SQL', 'Math']),
```

```
    (102, ['Java', 'SQL'])
```

```
])
```

```
career_reqs_rdd = sc.parallelize([
```

```
    ('Data Scientist', ['Python', 'SQL', 'Math', 'ML']),
```

```
    ('Software Engineer', ['Java', 'SQL', 'Git'])
```

```
])
```

```
# 2. Perform Distributed Cartesian Product to map users to roles
```

```
cartesian_rdd = user_skills_rdd.cartesian(career_reqs_rdd)
```

```
# 3. Map function to execute distributed set-difference and matching
```

```
def calculate_gap(pair):
```

```
(uid, u_skills), (role, r_skills) = pair
gaps = list(set(r_skills) - set(u_skills))
match_score = len(set(u_skills) & set(r_skills)) / len(r_skills)
return (uid, (role, match_score, gaps))
```

```
# 4. Filter and collect results above threshold
results_rdd = cartesian_rdd.map(calculate_gap).filter(lambda x: x[1][1] > 0.4)
print(results_rdd.collect())
```

Enterprise Scalability Additions & Explainable AI

- **Machine Learning Scaling via MLlib:** Utilizing PySpark MLlib's collaborative filtering (Alternating Least Squares), the system can analyze historical academic and career path data from millions of profiles to make highly accurate similarity-based recommendations.
- **Real-time Stream Ingestion (Spark Streaming + Kafka):** Dynamic job postings and course curricula updates are ingested via Apache Kafka. PySpark Structured Streaming continuously updates the Skills and Careers Databases to keep recommendations aligned with real-time market trends.
- **Probabilistic Logical Reasoning:** Bayesian Networks are integrated into the reasoning engine to replace rigid boolean logic rules. This allows the system to compute probabilistic success ratings under conditions of incomplete user profiles or uncertain career milestones.
- **Explainable AI (XAI) Integration:** By connecting the Knowledge Graph and the RAG-LLM pipeline to our Spark system, we output transparent justifications (e.g., 'This course is recommended because 91% of successfully placed Software Engineers in your region completed it'). This solves the core explainability gap identified in literature.